Get the Test Suite Running

At first glance this may strike you as a useless piece of advice. If the legacy team has been ignoring tests, nothing is there, right? Of course everything will run.

Well, not necessarily. There are at least two things you need to look out for. Even if the previous coders totally ignored tests, Rails may still have autogenerated test code. Fixtures or factories generated early in the project have moved out of date. If columns were deleted or renamed, fixtures won't load and you'll get errors galore. If validations have changed, factories might not work. It's only slightly less likely that the generated controller tests for a scaffold have drifted out of date with the code; if authentication and roles were added later, for example, you'll need to match that authentication in the tests.

In some ways your job is harder if the previous team flirted with writing tests and then gave up because they had not yet read this book (I'm assuming). In that case, you're likely to have all kinds of tests that may or may not have passed at one point and have been broken by later code—combined with inattention. You have to assume for the moment that the code is right and the tests are wrong—the exact opposite of a standard TDD scenario. (Mike Gunderloy presents a simple rule for initial triage of legacy tests in his *Rails Rescue Handbook*: if you can't figure out what a test is doing in five minutes, delete it.)

Take this opportunity to learn how the legacy code works, but do not change the code to match the tests at this point. If you can't figure out how to make a test pass, comment it out, mark it as pending, or delete it; add a note, and come back to it when you have a more thorough test scaffold in place. As I've discussed, test coverage is not the priority at this stage. The priority is a green test run that you can use as a base for future work.

I very tentatively recommend that if the code is on an older version of RSpec, one of the first things to try is an RSpec upgrade. There's an automated tool that moves from RSpec 2 to RSpec 3 syntax.[1] Upgrading to RSpec 3 gives you a few tools, such as `--only-failure`, that can be useful in a legacy codebase. That said, the specs need to be running without failure for the automatic upgrade to work. And you should be prepared to back out of this upgrade quickly if there are problems. If you don't know how the tests work and the upgrade gives you a lot of failures, now is not the time to be guessing.

1. http://rspec.info/upgrading-from-rspec-2